# Reconocimiento de audio musical mediante la Transformada de Fourier: huella acústica y demo web full stack

Andrés Felipe Verbel Pacheco
*Curso de Desarrollo Web Full Stack, CUC*
*Octubre de 2026*

***Resumen*—Este trabajo investiga cómo se reconoce una canción a partir de un fragmento de audio mediante la Transformada de Fourier. Se presentan la transformada discreta (DFT), el algoritmo FFT de Cooley y Tukey, la ventana de Hann y el espectrograma, y sobre ellos un sistema de huella acústica basado en una «constelación» de picos: se detectan máximos locales del espectrograma, se combinan en pares para formar hashes de 24 bits y la canción se identifica por votación del desplazamiento temporal. El sistema se implementó como una aplicación web full stack (Node.js, Express y Web Audio API) con un espectrograma 3D interactivo, y se evaluó con una base de 12 canciones sintéticas de 30 s. Con fragmentos de 5 s, la canción correcta fue la primera candidata en 100 % de los casos con ruido blanco de −5 dB de relación señal-ruido y en 87 % con −10 dB; ninguna de 360 consultas con canciones ajenas a la base superó el umbral de decisión. Para N = 4096, la FFT fue 258 veces más rápida que la DFT directa.**

***Palabras clave*—Transformada de Fourier, FFT, espectrograma, huella acústica, reconocimiento de música, hashing, desarrollo web full stack.**

## I. INTRODUCCIÓN

Identificar una canción a partir de unos segundos de audio, grabados con un micrófono y en presencia de ruido, es el problema que resuelven servicios como Shazam [1]. La dificultad es doble: la señal no se puede comparar muestra a muestra con una grabación almacenada, porque el fragmento empieza en un instante arbitrario y está degradado; y la búsqueda debe ser rápida en bases de millones de piezas. La solución habitual consiste en extraer de la señal una *huella acústica* compacta y robusta [2], [3], y buscarla en un índice.

La Transformada de Fourier es la herramienta que permite pasar del dominio del tiempo al de la frecuencia, donde una nota musical se manifiesta como un conjunto de picos estables. Este informe responde a la actividad de investigar el funcionamiento del reconocimiento de audio (música) y el uso de la transformada de Fourier. Sus aportes son: (i) la presentación de los modelos matemáticos (DFT, FFT, ventanas, espectrograma, hashing y votación); (ii) una implementación en JavaScript con pruebas automáticas; (iii) una evaluación experimental de la velocidad de la FFT, la exactitud frente al ruido y los falsos positivos; y (iv) una demo web full stack con captura de audio, espectrograma interactivo y reconocimiento.

## II. MARCO TEÓRICO

### A. Muestreo

Una señal de audio continua $x(t)$ se representa por muestras $x[n] = x(n/f_s)$. Según el teorema del muestreo [4], solo se pueden representar sin ambigüedad las frecuencias menores que $f_s/2$. Aquí se trabaja con $f_s = 8.000$ Hz (banda de 0 a 4 kHz), suficiente para la melodía y los primeros armónicos y barata de procesar; cualquier audio de entrada se remuestrea a esa frecuencia.

### B. Transformada discreta de Fourier

Para un bloque de $N$ muestras, la DFT calcula la amplitud y fase de $N$ componentes sinusoidales [5]:

$$X[k] = \sum_{n=0}^{N-1} x[n]\, e^{-j2\pi kn/N}, \qquad k = 0, \dots, N-1. \tag{1}$$

El índice $k$ corresponde a la frecuencia $f_k = k\, f_s/N$, por lo que la resolución es $\Delta f = f_s/N$. Como $x[n]$ es real, $X[N-k] = X[k]^*$ y basta con los $N/2$ primeros coeficientes. Evaluar (1) directamente cuesta $\Theta(N^2)$ operaciones.

### C. Transformada rápida de Fourier

El algoritmo de Cooley y Tukey [6] separa las muestras pares e impares. Si $E[k]$ y $O[k]$ son las DFT de tamaño $N/2$ de cada grupo y $W_N = e^{-j2\pi/N}$, entonces

$$\begin{aligned} X[k] &= E[k] + W_N^k\, O[k], \\ X[k+N/2] &= E[k] - W_N^k\, O[k], \end{aligned} \qquad 0 \le k < N/2. \tag{2}$$

Aplicada recursivamente (con $N$ potencia de 2), la recurrencia $T(N) = 2T(N/2) + \Theta(N)$ da un costo $\Theta(N \log_2 N)$. La implementación usada es iterativa, en el lugar, con reordenamiento por inversión de bits y factores de giro precalculados.

### D. Ventana, espectrograma y resolución

Para analizar música, cuyo contenido cambia en el tiempo, se divide la señal en bloques solapados, cada uno multiplicado por una ventana que atenúa los bordes y reduce la fuga espectral [7]. Se usa la ventana de Hann y la transformada de tiempo corto (STFT) con salto de $H$ muestras:

$$\begin{aligned} w[n] &= \tfrac{1}{2} - \tfrac{1}{2}\cos\tfrac{2\pi n}{N}, \quad 0 \le n < N, \\ X[m,k] &= \sum_{n=0}^{N-1} w[n]\, x[mH+n]\, e^{-j2\pi kn/N}, \\ S[m,k] &= 10\log_{10}|X[m,k]|^2. \end{aligned} \tag{3}$$

La matriz $S[m,k]$, en dB, es el *espectrograma*. Con $N = 1024$, $H = 256$ y $f_s = 8.000$ Hz, cada ventana dura 128 ms, el salto es de 32 ms y la resolución en frecuencia es $\Delta f = 7,81$ Hz por bin. Una ventana más larga mejora $\Delta f$ pero empeora la resolución temporal: ambas no pueden ser arbitrariamente pequeñas a la vez [8].

### E. Estructura de una nota musical

En la escala temperada, la nota MIDI $n$ tiene frecuencia fundamental $f_0 = 440 \cdot 2^{(n-69)/12}$ Hz. Un instrumento produce además armónicos, de modo que una nota es una suma $x(t) = \sum_h a_h \sin(2\pi h f_0 t)$, que en el espectrograma aparece como líneas horizontales en $f_0, 2f_0, 3f_0, \ldots$ que duran lo que la nota. Esas líneas son las estructuras que la huella aprovecha.

## III. MODELO DE HUELLA ACÚSTICA

El método sigue el enfoque de constelación de picos de Wang [1]. La Fig. 1 resume la tubería: la señal se convierte en espectrograma; de él se extraen picos; los picos se combinan en pares que producen hashes; y la canción se determina contando coincidencias consistentes en el índice.

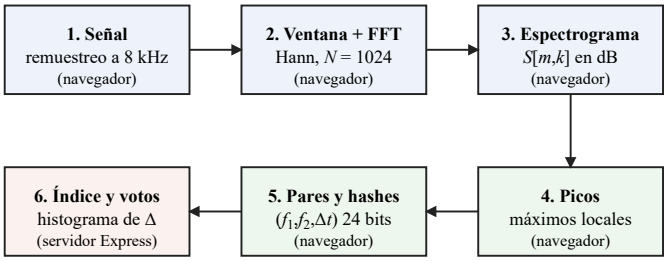



Fig. 1. Tubería de reconocimiento. Las etapas 1–5 se ejecutan en el navegador y solo viajan los hashes; la etapa 6 se ejecuta en el servidor.

### A. Detección de picos

Un punto $(m, k)$ del espectrograma es un pico si es el máximo de su vecindad de $\pm\Delta_t$ ventanas y $\pm\Delta_f$ bins y, además, está a menos de $\gamma$ dB del máximo global $S_{\max}$:

$$(m,k) \text{ es pico} \iff S[m,k] = \max_{|m'-m|\le\Delta_t,\,|k'-k|\le\Delta_f} S[m',k'] \;\wedge\; S[m,k] > S_{\max} - \gamma. \tag{4}$$

El criterio de máximo local reparte los picos de manera uniforme en el plano tiempo-frecuencia y es insensible a la ganancia global, pues compara dB entre sí. El margen $\gamma$ descarta lo que queda por debajo del ruido (Sección V-E). La Fig. 2 muestra el resultado sobre 6 s de una canción de la base: 31 picos (5,3 por segundo).

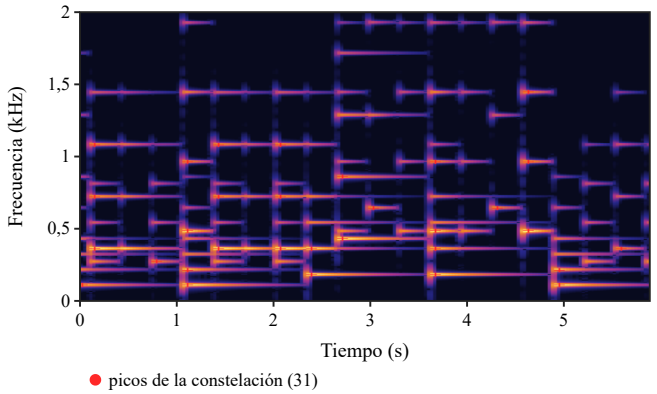


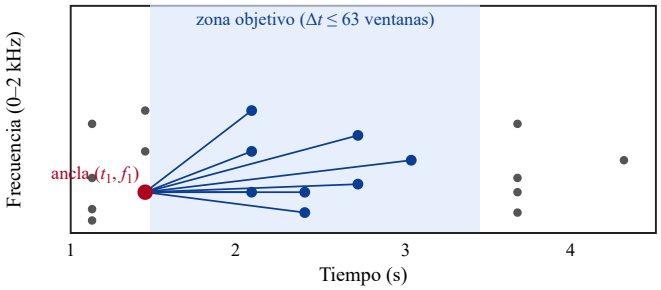

Fig. 2. Espectrograma (0–2 kHz) de un fragmento del Tema 04 y su constelación de picos. Las líneas horizontales son las notas y sus armónicos.

### B. Pares de picos y hashes

Un solo pico no es suficientemente selectivo. Por eso cada pico *ancla* se une con los siguientes $F = 10$ picos de una zona objetivo (hasta 63 ventanas después, Fig. 3) y cada par define un hash de 24 bits con las frecuencias de ambos picos y su separación en el tiempo:

$$h = (k_1 \cdot 512 + k_2) \cdot 64 + \Delta m, \qquad \Delta m = m_2 - m_1 \in [1, 63].$$

Ese hash no depende del instante absoluto del par, solo de la diferencia $\Delta m$, y por eso sirve para fragmentos que empiezan en cualquier punto. La fuente de selectividad es combinatoria: dos frecuencias (9 bits cada una) y un intervalo (6 bits) aportan 24 bits frente a los 9 de un pico aislado.




Cada par (ancla, objetivo) genera un hash: ($f_1$, $f_2$, $\Delta t$) = 1510292

Fig. 3. Pares ancla-objetivo. Cada línea azul es un par que genera un hash; la zona sombreada es la zona objetivo.

### C. Índice invertido y votación

Durante el registro, cada hash de cada canción se guarda con su identificador y su instante $m_{\text{bd}}$. En la consulta, cada hash $(h, m_{\text{q}})$ recupera las entradas con el mismo $h$ y vota por el desplazamiento $\delta = m_{\text{bd}} - m_{\text{q}}$. Si el fragmento pertenece a la canción, muchos pares coinciden con el *mismo* $\delta$ (la posición del fragmento); las coincidencias fortuitas se reparten entre desplazamientos distintos:

$$\text{puntaje}_s = \max_{\delta} \left[ c_s(\delta - 1) + c_s(\delta) + c_s(\delta + 1) \right], \tag{6}$$

donde $c_s(\delta)$ es el número de coincidencias de la canción $s$ con desplazamiento $\delta$. Se suman tres bins consecutivos porque el fragmento empieza en una muestra arbitraria y los instantes se cuantizan en pasos de 32 ms, lo que desplaza algunos picos una ventana. La canción ganadora es $\hat{s} = \arg\max_s \text{puntaje}_s$ y se acepta solo si el puntaje alcanza un umbral $\tau = 12$; además, $\delta H / f_s$ estima la posición del fragmento en la canción.

### D. Ruido y colisiones

Para evaluar la robustez se añade ruido blanco gaussiano de varianza $\sigma^2$ fijada por la relación señal-ruido respecto a la potencia $P_x$ del fragmento:

$$\text{SNR}_{\text{dB}} = 10 \log_{10} \frac{P_x}{\sigma^2} \;\Rightarrow\; \sigma^2 = \frac{P_x}{10^{\text{SNR}_{\text{dB}}/10}}. \tag{7}$$

Si los $U$ hashes distintos del índice estuvieran repartidos uniformemente en $2^{24}$ valores, un hash cualquiera coincidiría por azar con probabilidad $p \approx U/2^{24}$. En música real y en tonos sintéticos la distribución no es uniforme, porque las notas ocupan pocas frecuencias y los ritmos pocos intervalos. En la Sección V-D se mide ese efecto.

## IV. IMPLEMENTACIÓN FULL STACK

### A. Arquitectura

Como en los servicios comerciales, el cálculo costoso se hace en el cliente y el servidor solo busca. El **frontend** (HTML, CSS y JavaScript) obtiene el audio de tres fuentes: un fragmento de la base con ruido ajustable, un archivo del usuario o el micrófono, y lo decodifica y remuestrea a 8 kHz con la Web Audio API [9]. Luego calcula el espectrograma, los picos y los hashes con el módulo `huella.js` y dibuja el espectrograma en 3D con Three.js [10] (superficie con la potencia como altura y color, picos, pares de hash y cabezal de reproducción) y en 2D sobre un `canvas`. El **backend** es una API REST en Node.js y Express [11] que, al iniciar, lee las canciones, calcula sus huellas y construye el índice en memoria; recibe los hashes, valida las entradas (enteros de 24 bits, límite de tamaño) y responde con los candidatos, la posición estimada y el histograma de votos. El mismo módulo `huella.js` corre en Node y en el navegador, de modo que cliente y servidor calculan huellas idénticas.

### B. API y parámetros

La API expone `GET /api/canciones`, `GET /api/estado` y `POST /api/reconocer`, que recibe `{"huellas": [[hash, tiempo], …]}`. La Tabla I resume los parámetros de la huella.

TABLA I
Parámetros de la huella acústica

| *Parámetro* | *Valor* |
|---|---|
| Frecuencia de muestreo $f_s$ | 8.000 Hz |
| Ventana / FFT ($N$), salto ($H$) | 1024 / 256 muestras |
| Vecindad de pico ($\Delta_t$, $\Delta_f$) | ±8 ventanas, ±10 bins |
| Margen bajo el máximo global ($\gamma$) | 12 dB |
| Pares por ancla ($F$), $\Delta m$ máximo | 10, 63 ventanas |
| Umbral de decisión ($\tau$) | 12 votos |

### C. Base de canciones de prueba

Para no usar material con derechos de autor, la base contiene 12 piezas sintéticas de 30 s generadas por código con semillas fijas: una melodía en una escala (mayor, menor, pentatónica o dórica) con estructura de frases repetidas A–B–A–C, un arpegio y un bajo, cada nota con cuatro armónicos y envolvente de ataque y decaimiento. Cada pieza tiene tonalidad, tempo (80–139 bpm), timbre y afinación (±50 centésimas) propios; esta última variación se añadió porque, sin ella, piezas distintas compartían frecuencias exactas y generaban coincidencias fortuitas poco realistas.

### D. Verificación

El módulo se verificó con 13 pruebas automáticas (`node:test`): la FFT coincide con la DFT directa; una senoidal en el bin $k$ produce picos en $k$ y $N-k$; se cumple el teorema de Parseval; la ventana de Hann toma los valores esperados; el pico del espectrograma de una nota de 440 Hz cae en el bin correcto; los hashes caben en 24 bits; se reconocen fragmentos ruidosos de 5 s; una canción ajena no supera el umbral; y la posición estimada de un fragmento en una pieza sin repeticiones tiene error menor que 0,1 s. Todas pasan.

## V. RESULTADOS

### A. FFT frente a DFT directa

La Tabla II y la Fig. 4 comparan el tiempo de una transformada de $N$ puntos con la FFT implementada y con la DFT directa (ambas en Node.js, entrada compleja aleatoria). El error máximo entre las dos es menor que $6{,}2\times10^{-13}$, es decir, del orden del redondeo de doble precisión. La ventaja de la FFT crece con $N$, como predice $N/\log_2 N$, y llega a 258× para $N = 4096$. Con la ventana usada ($N = 1024$), una FFT toma unas decenas de microsegundos.

TABLA II
Tiempo de la FFT y la DFT directa

| *N* | *FFT (ms)* | *DFT (ms)* | *Acel.* | *Error máx.* |
|---|---|---|---|---|
| 64 | 0,0019 | 0,017 | 9× | $7{,}1\times10^{-15}$ |
| 128 | 0,0036 | 0,048 | 13× | $1{,}4\times10^{-14}$ |
| 256 | 0,0066 | 0,188 | 28× | $2{,}1\times10^{-14}$ |
| 512 | 0,0127 | 0,629 | 49× | $5{,}0\times10^{-14}$ |
| 1024 | 0,0287 | 2,521 | 88× | $1{,}1\times10^{-13}$ |
| 2048 | 0,0670 | 10,462 | 156× | $2{,}9\times10^{-13}$ |
| 4096 | 0,1534 | 39,535 | 258× | $6{,}2\times10^{-13}$ |

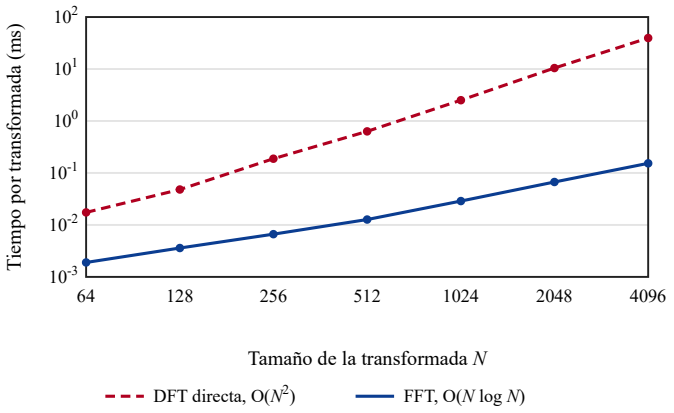



Fig. 4. Tiempo por transformada en función de $N$ (escala log-log).

### B. Votación por desplazamiento

La Fig. 5 muestra el histograma de votos para un fragmento de 5 s del Tema 7 con ruido de 5 dB. La canción correcta acumula 194 votos en un único desplazamiento (el instante real del fragmento), mientras que la mejor canción rival no pasa de 1. Los picos menores de la canción correcta corresponden a otras apariciones de la misma frase musical, que se repite exactamente en la pieza; por eso, en piezas con repeticiones, la posición estimada es ambigua.

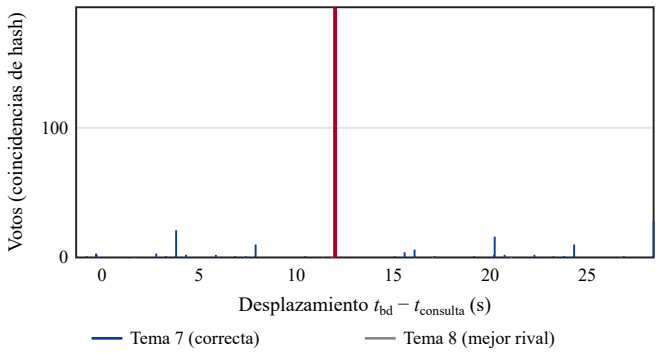



Fig. 5. Histograma de desplazamientos para una consulta de 5 s con SNR de 5 dB.

### C. Exactitud frente al ruido y a la duración

Se evaluó el sistema con 12 canciones, 5 fragmentos aleatorios por canción y condición (60 consultas por celda), con duraciones de 2 a 10 s y SNR entre ∞ (sin ruido) y −15 dB. La Tabla III muestra, para cada condición, el porcentaje en que la canción correcta fue la primera candidata (top-1) y el porcentaje en que, además, se superó el umbral; la Fig. 6 grafica el top-1. Con 5 s o más se alcanza 100 % de top-1 hasta −5 dB; con 5 s, el top-1 baja a 87 % con −10 dB y a 63 % con −15 dB, y con 10 s a 98 % y 77 %. Los fragmentos de 2 s se identifican (100 % sin ruido) pero su puntaje es bajo, de modo que solo 78 % supera el umbral; con 3 s ya lo supera 97 %. El puntaje mediano sin ruido es de 88 votos para 5 s y 179 para 10 s: crece con la duración, pues hay más pares.

TABLA III
Top-1 / aceptadas (%) según SNR (dB) y duración

| *Dur.* | ∞ | *20* | *10* | *0* | *-5* | *-10* | *-15* |
|---|---|---|---|---|---|---|---|
| 2 s | 100 / 78 | 100 / 62 | 100 / 67 | 98 / 62 | 98 / 52 | 57 / 0 | 38 / 0 |
| 3 s | 100 / 97 | 100 / 92 | 100 / 97 | 100 / 98 | 100 / 90 | 83 / 0 | 47 / 0 |
| 5 s | 100 / 100 | 100 / 100 | 100 / 100 | 100 / 100 | 100 / 98 | 87 / 12 | 63 / 0 |
| 10 s | 100 / 100 | 100 / 100 | 100 / 100 | 100 / 100 | 100 / 100 | 98 / 43 | 77 / 0 |

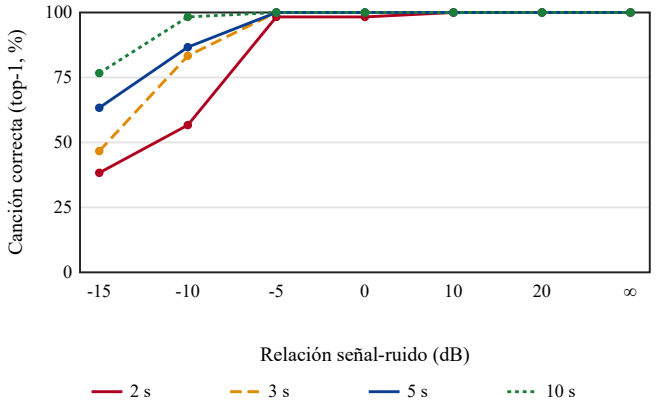



Fig. 6. Porcentaje de aciertos (top-1) frente a la relación señal-ruido.

### D. Falsos positivos

Se consultó la base con fragmentos de 12 canciones *no* incluidas, en seis condiciones (Tabla IV). De 360 consultas hubo 0 falsos positivos; el puntaje máximo de una canción ajena fue 6 votos, por debajo del umbral $\tau = 12$ (que se fijó en el doble de ese máximo). Aun así, 2,0 % de los hashes de una canción ajena existen en el índice, 48 veces más que el 0,042 % que daría una distribución uniforme, lo que confirma la no uniformidad descrita en la Sección III: la decisión no se puede basar en que «haya coincidencias», sino en que se concentren en un desplazamiento.

TABLA IV
Consultas con canciones ajenas a la base

| *Dur.* | *SNR* | *Consultas* | *Falsos +* | *Punt. máx.* |
|---|---|---|---|---|
| 2 s | ∞ | 60 | 0 | 4 |
| 2 s | 0 dB | 60 | 0 | 4 |
| 5 s | ∞ | 60 | 0 | 5 |
| 5 s | 0 dB | 60 | 0 | 5 |
| 10 s | ∞ | 60 | 0 | 6 |
| 10 s | 0 dB | 60 | 0 | 6 |

### E. Sensibilidad al margen de picos

El parámetro $\gamma$ resultó decisivo (Tabla V). Con márgenes grandes (30 dB) se conservan muchos picos débiles, que el ruido reemplaza o desplaza, y el top-1 a −5 dB baja a 83 %, además de aumentar el puntaje de canciones ajenas a 15. Con márgenes pequeños, la constelación contiene solo los picos más fuertes, que sobreviven al ruido y producen menos hashes (34 por segundo con $\gamma = 12$ dB). La elección se hizo con canciones sintéticas, cuyas notas tienen una amplitud parecida; música real, con gran dinámica, requeriría un umbral adaptativo por zonas en lugar de uno global.

TABLA V
Efecto del margen γ (fragmentos de 5 s)

| *γ (dB)* | *Hashes/s* | *0 dB (%)* | *−5 dB (%)* | *Punt. ajeno máx.* |
|---|---|---|---|---|
| 8 | 16 | 100 | 100 | 2 |
| 12 | 34 | 100 | 100 | 3 |
| 20 | 69 | 100 | 89 | 4 |
| 30 | 102 | 100 | 83 | 15 |
| 45 | 113 | 100 | 83 | 15 |

### F. Costo y demo funcional

Calcular la huella de un fragmento de 5 s toma unos 14 ms en el cliente y la búsqueda en el servidor 0,09 ms, con 12.359 huellas (7.045 hashes distintos) de 12 canciones; construir el índice al iniciar el servidor tarda 852 ms. La búsqueda recorre solo las listas de los hashes de la consulta, por lo que crece con el número de consultas y la longitud media de las listas (2,1 entradas por hash coincidente aquí), no con el número de canciones. La Fig. 7 muestra la interfaz: las cinco etapas del algoritmo se animan durante el reconocimiento y pueden consultarse una a una, el espectrograma 3D se puede girar y acercar, y hay modo claro y oscuro.

## VI. DISCUSIÓN Y LIMITACIONES

- La base es sintética y pequeña (12 piezas). Música real tiene voz, percusión, reverberación y compresión; los resultados absolutos no son extrapolables a un servicio comercial.
- El método no es invariante a cambios de tempo ni de tono, que alteran $\Delta m$ y $k$; tampoco se evaluó ruido no blanco ni distorsión de altavoz y micrófono.
- Los umbrales ($\gamma$, $\tau$) se ajustaron con las mismas familias de piezas; deberían recalibrarse con material nuevo.
- La captura con micrófono depende del navegador y de los permisos del usuario; se implementó, pero la evaluación cuantitativa se hizo con ruido sintético.

- El índice vive en memoria. Para millones de canciones se necesitaría una base de datos con índice sobre el hash y particionado.

## VII. CONCLUSIONES

La Transformada de Fourier convierte un fragmento de audio en un espectrograma donde las notas son estructuras estables, y a partir de ahí se puede construir una huella robusta: picos locales, pares de picos, hashes e histograma de desplazamientos. La FFT hace práctico el cálculo, con una aceleración de 258 veces sobre la DFT directa para $N = 4096$. En las pruebas, fragmentos de 5 s se identificaron en 100 % de los casos con ruido de $-5$ dB y ninguna canción ajena fue aceptada. La demo full stack integra todo el proceso, con el cálculo pesado en el navegador y la búsqueda en el servidor.

Como trabajo futuro se propone evaluar con música real y con grabaciones de micrófono, incorporar un umbral de picos adaptativo, estudiar la invariancia a cambios de tempo y tono, y almacenar el índice en una base de datos persistente.




Fig. 7. Demo web: fragmento del Tema 07 con ruido de 5 dB. Espectrograma 3D con picos y pares de hash, resultado con medidor de confianza, forma de onda e histograma de desplazamientos.

## REFERENCIAS


[1] A. Wang, "An industrial-strength audio search algorithm," in *Proc. 4th Int. Conf. Music Information Retrieval (ISMIR)*, Baltimore, MD, USA, Oct. 2003, pp. 7–13.

[2] J. Haitsma and T. Kalker, "A highly robust audio fingerprinting system," in *Proc. 3rd Int. Conf. Music Information Retrieval (ISMIR)*, Paris, France, Oct. 2002, pp. 107–115.

[3] P. Cano, E. Batlle, T. Kalker, and J. Haitsma, "A review of audio fingerprinting," *J. VLSI Signal Process. Syst.*, vol. 41, no. 3, pp. 271–284, Nov. 2005.

[4] C. E. Shannon, "Communication in the presence of noise," *Proc. IRE*, vol. 37, no. 1, pp. 10–21, Jan. 1949.

[5] A. V. Oppenheim and R. W. Schafer, *Discrete-Time Signal Processing*, 3rd ed. Upper Saddle River, NJ, USA: Pearson, 2010.

[6] J. W. Cooley and J. W. Tukey, "An algorithm for the machine calculation of complex Fourier series," *Math. Comput.*, vol. 19, no. 90, pp. 297–301, Apr. 1965.

[7] F. J. Harris, "On the use of windows for harmonic analysis with the discrete Fourier transform," *Proc. IEEE*, vol. 66, no. 1, pp. 51–83, Jan. 1978.

[8] D. Gabor, "Theory of communication," *J. Inst. Electr. Eng.*, vol. 93, no. 26, pp. 429–457, 1946.

[9] P. Adenot and R. Toy, Eds., "Web Audio API," W3C Recommendation, Jun. 2021. [Online]. Available: https://www.w3.org/TR/webaudio/

[10] "three.js: JavaScript 3D library." [Online]. Available: https://threejs.org

[11] "Express: Fast, unopinionated, minimalist web framework for Node.js." [Online]. Available: https://expressjs.com